Multi-Line Insurance Policy and Claims Management System

Problem Statement:

An insurance carrier, which manages vehicle, property, and life insurance, currently uses a legacy system that results in manual, slow, and inconsistent policy quoting, issuance, and claims handling. The lack of a unified platform makes cross-selling difficult, and the manual claims review process leads to long resolution times, high operational costs, and poor customer experience.

The business needs a single Salesforce solution to:

1. Standardize and accelerate the quoting and policy issuance process for all insurance lines.
2. Automate claim initiation and routing based on policy type and claim severity.
3. 3. Provide a 360-degree view of the Customer, Policy, and Claim data in one place.

Objectives:

● Design a flexible data model capable of storing diverse product data (e.g., vehicle VIN for Vehicle, square footage for Property).

● Implement Screen Flows to guide agents through complex, multi-step quoting processes for different policy types.

● Develop a dynamic Apex/LWC Claims Dashboard for adjusters to efficiently manage their workload.

● Automate claims approval/escalation based on claim amount and policy history using Record-Triggered Flows.

● Enforce data validation across all insurance products to ensure compliance and accuracy.

Challenges:

● Variable Data Management: Handling the diverse fields required for Vehicle, Property, and Life insurance within a unified object structure using Record Types/Field Sets.

● Approval Complexity: Designing a Claims Approval Flow that dynamically routes based on policy type, claim amount, and adjuster level.

● API Integration Simulation (Apex): Simulating a callout to an external motor vehicle or property database for quote verification (requires mock test classes).

● User Adoption: Creating intuitive Screen Flows that minimize clicks and maximize agent efficiency during the quoting process.

Solution Approach:

The solution leverages Screen Flows for interactive quoting, a flexible Custom Data Model for policy diversity, and LWC/Apex for the high-performance Claims Management dashboard.

1. Flexible Data Model: Create Policy and Claim objects with different Record Types for Vehicle, Property, and Life.

2. Quoting Flows (Declarative): Build multi-screen Screen Flows to capture policy-specific data efficiently.

3. Claims Logic (Programmatic/Flow): Use Apex for complex rating logic and Record-Triggered Flow for automated routing.

4. UI Development (LWC): Create a highly focused Claims Management dashboard for adjusters.

Milestones:

Milestone 1: Core Data Model & Policy Configuration:

User Story1: Creation of custom objects:

1. Create Policy custom object:

● Go to setup, Click on object Manager.

● Click on create and select custom object.

● Fill the following details:

● Label: Policy

● Plural Label: Policies

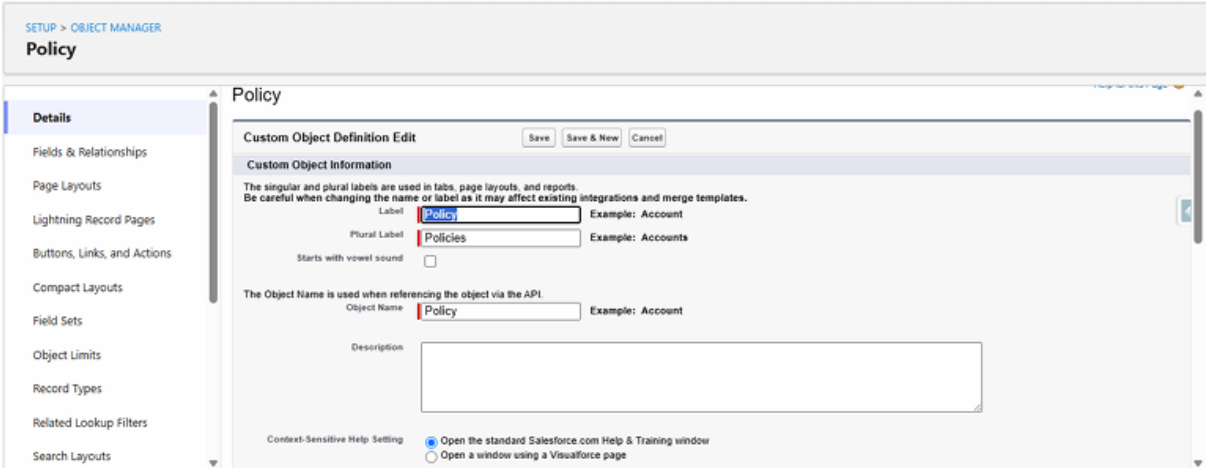
● Object Name: Auto Populated

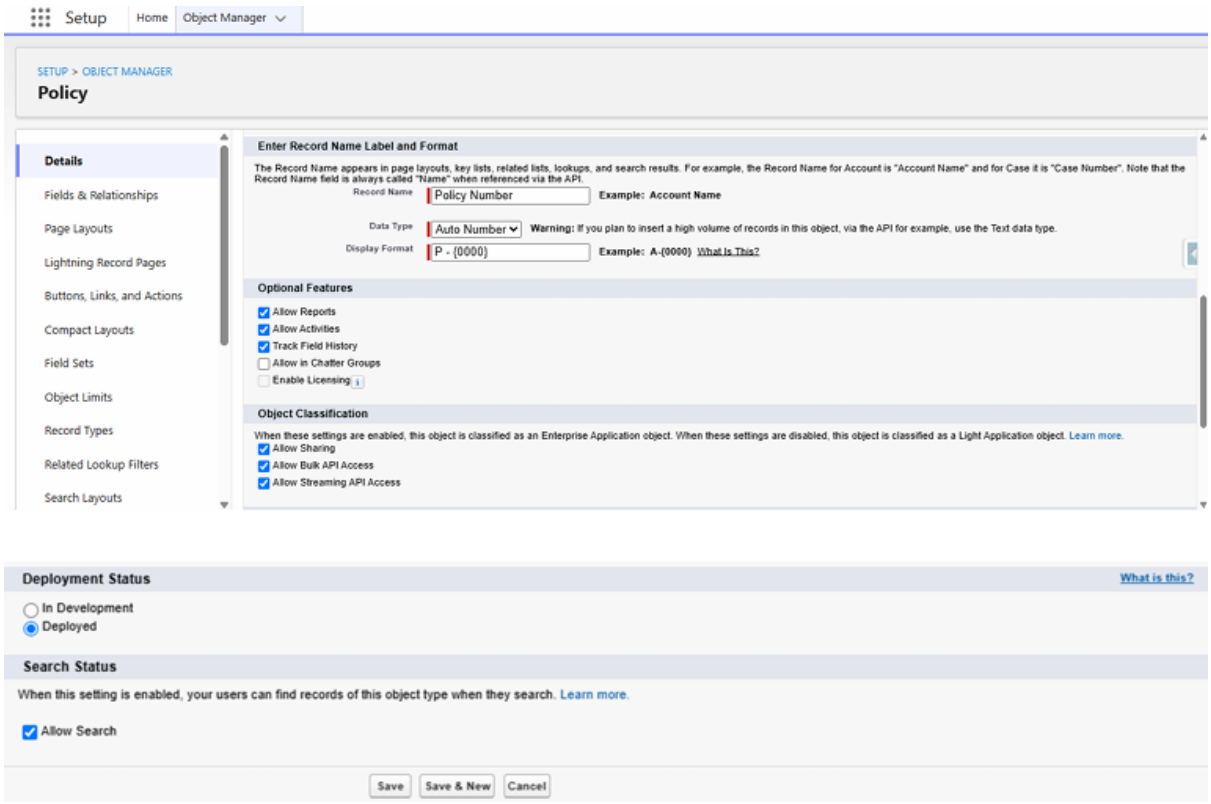
● Record Name: Policy Number

● Data type: Auto Number

● Format: P - {0000}

- Starting Number: 1
- Under optional features:
- Allow Reports, Activities and Track field History
- Under Deployment Status:
- Select the deployed radio button.
- Under Search Status:
- Check the Allow Search checkbox.
- Click on save.





2.Define Record Types on Policy Object:

○ Go to setup, Click on Object Manager.

○ Search for Policy objects and click on Policy__c.

○ In the left side panel, click on the record types.

○ Click on New.

○ Fill out the following details:

○ Select the existing record type as Master.

○ Give record type label as “Auto”

○ Record Type Name will be auto populated.

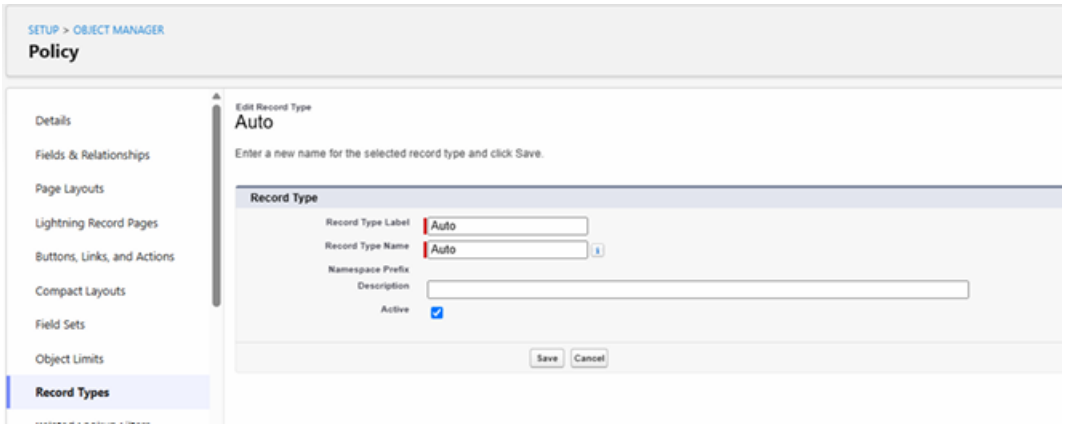
○ Give a meaningful description.

○ Check the active checkbox

○ Check the Make Available checkbox for all the profiles.

○ Click on Next.

● It will redirect to the Assign page layouts page, Leave everything as is, and click on save.



Repeat the same steps for creating the property and life record types as well.

3.Create Field Sets for policy specific data:

○ Go to setup, Click on Object Manager.

○ Search for Policy object and click on Policy__c.

○ In the left side panel, click on the Field Sets.

○ Click on New.

○ Fill out the following details:

○ Field Set Label as Vehicle

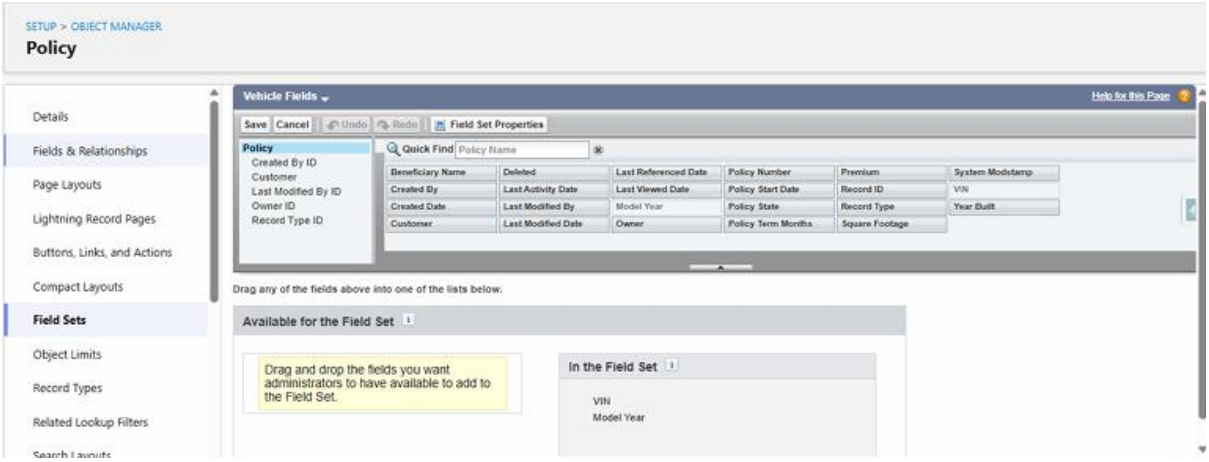
○ Field Set Name will be Auto populated.

○ Where is this used: In the policy object.

○ Click on save.

○ Click on the field set you have just created and it will redirect to the page layout setup.

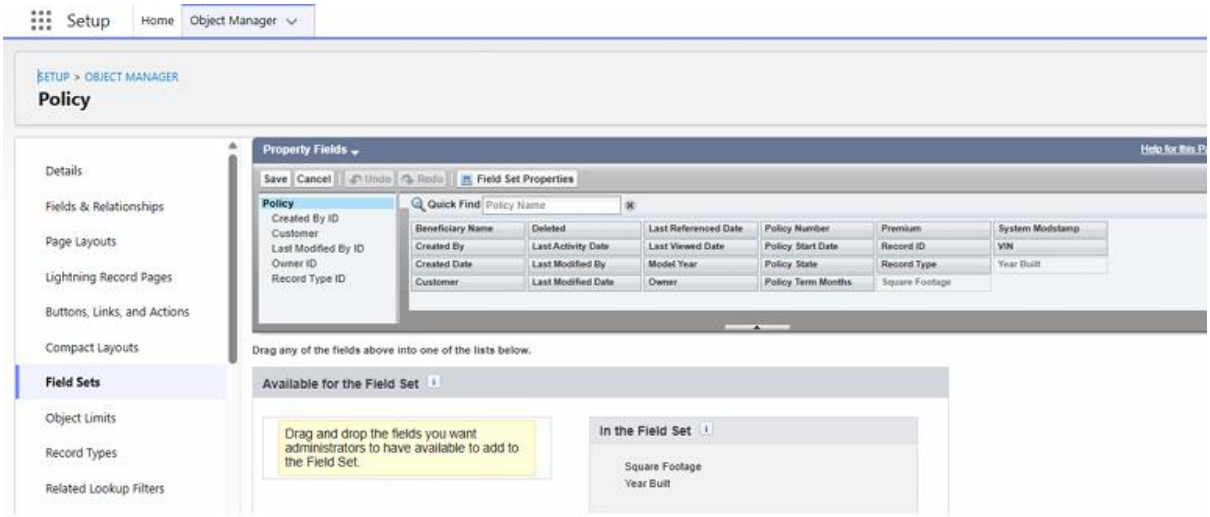
○ Drag VIN, and model year fields to the field set and click on save.



Repeat the same steps for creating the property and life Field sets as well.

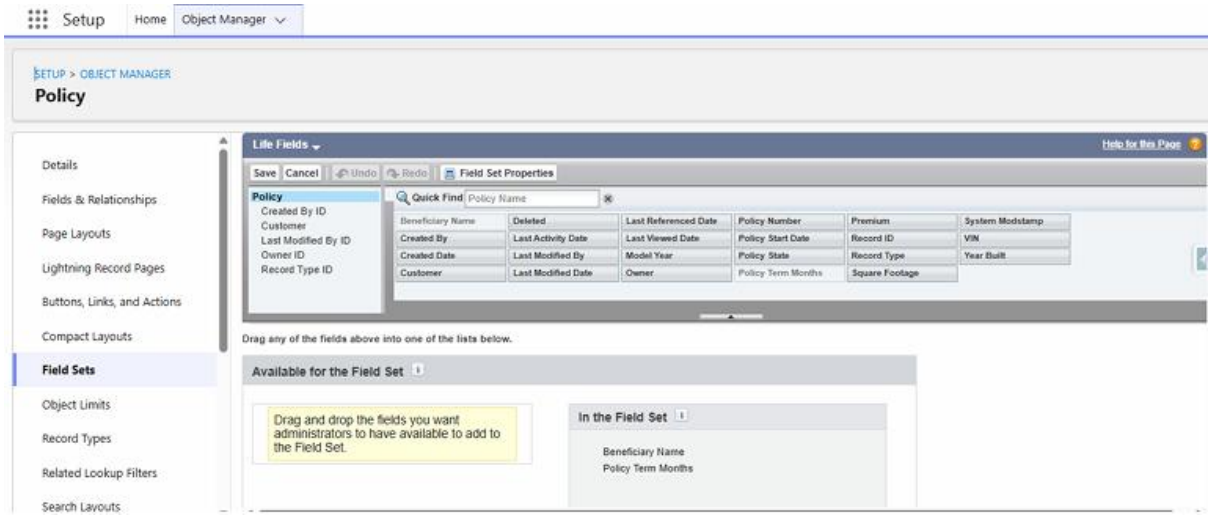
3.1Property FieldSet Requires:

- Square footage
- Year Built



3.2 Life FieldSet Requires:

- Beneficiary Name
- Policy Term Months



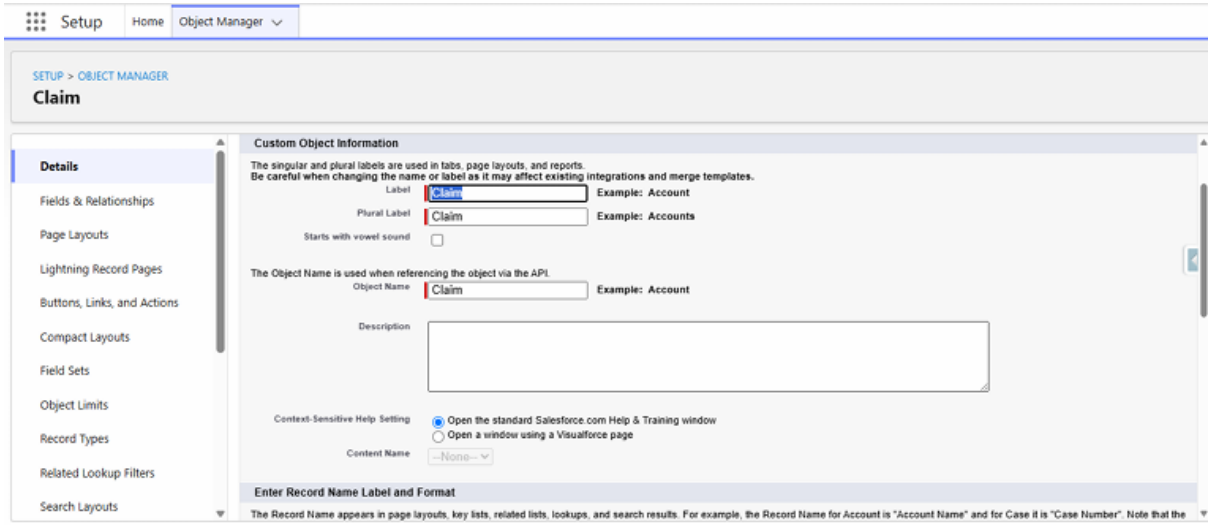
4. Create Fields in the Policy Object:

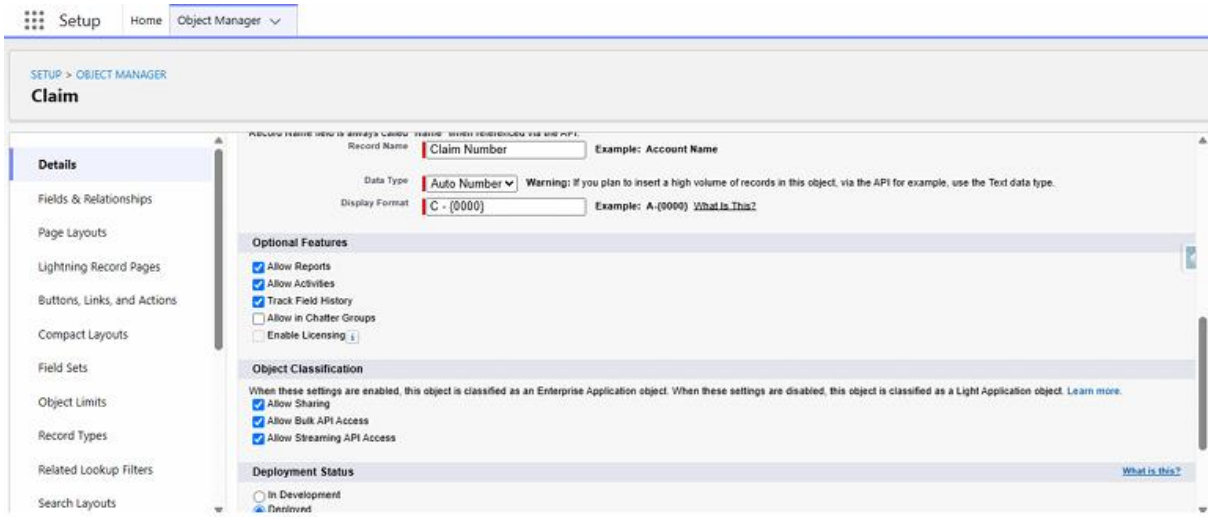
S.No	Name of the Field	Data Type	Required
1	Beneficiary Name	Text(50)	
2	Customer	Lookup(Contact)	
3	Model Year	Text(5)	
4	Policy Start Date	Date	
5	Policy Term Months	Number(18, 0)	
6	Premium	Currency(16, 2)	
7	Square Footage	Number(18, 0)	
8	VIN	Text(20)	
9	Year Built	Text(5)	

5.Create Claim custom object:

- Go to setup, Click on object Manager.
- Click on create and select custom object.
- Fill the following details:
- Label: Claim
- Plural Label: Claim
- Object Name: Auto Populated
- Record Name: Claim Number
- Data type: Auto Number
- Format: C - {0000}

- Starting Number: 1
- Under optional features:
- Allow Reports, Activities and Track field History
- Under Deployment Status:
- Select the deployed radio button.
- Under Search Status:
- Check the Allow Search checkbox.
- Click on save.





6. Define Record Types on Claim Object:

○ Go to setup, Click on Object Manager.

○ Search for Claim object and click on Claim__c.

○ In the left side panel, click on the record types.

○ Click on New. ○ Fill out the following details:

○ Select the existing record type as Master.

○ Give record type label as “Accident”

Record Type Name will be auto populated.

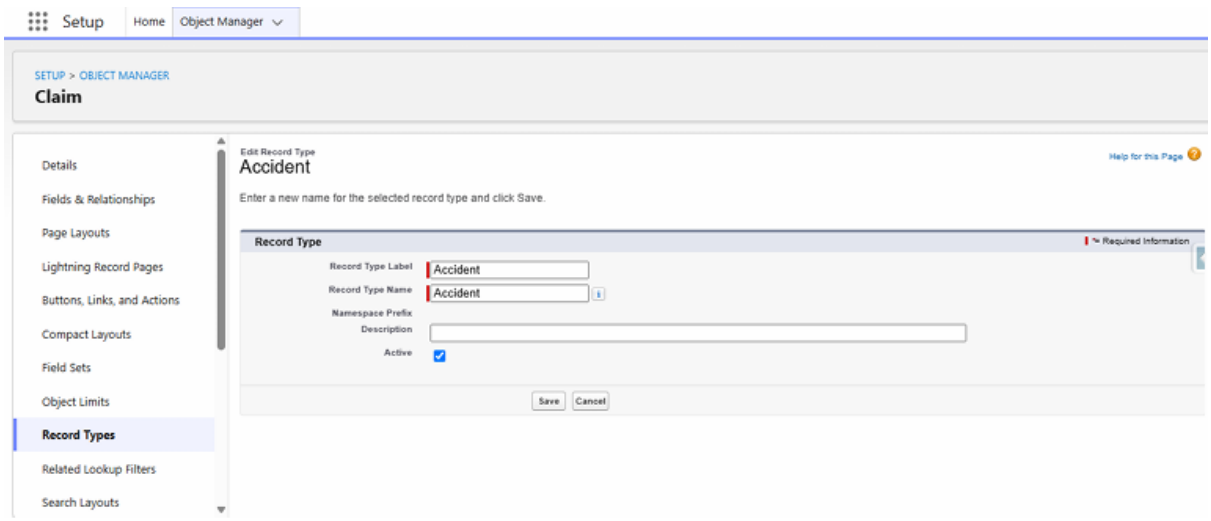
○ Give a meaningful description.

○ Check the active checkbox.

○ Check the Make Available checkbox for all the profiles.

○ Click on Next.

● It will redirect to the Assign page layouts page, Leave everything as is, and click on save.



//Repeat the same steps for creating the property and life record types as well.

7. Create Fields in the Claim Object:

S.No	Field Name	Data Type	Required
1	Adjuster	Lookup(User)	
2	Approval Status	Picklist(New, Submitted for	

		Approval, Approved, Rejected)	
3	**Claim Amount**	**Currency(16, 2)**	
4	**Date of Loss**	**Date/Time**	
5	**Description**	**Long Text Area**	
6	**Policy**	**Lookup(Policy)**	

8. Create an AutoQuoting Screen Flow

● Go to setup, Search for flows in the quick find box.

● Click on flows and click on new flow.

● Select Screen Flow and Click on Create.

● Add Get Records Element immediately after the start element. ● Give Label as Get Vehicle RT ID

● Api Name will be auto populated

● Select Record Type for Object

● For Conditional Requirements, Select All Conditions are met

● The Conditions include:

● sObject Type Name Equals Policy__c

● Record Type Name Equals Auto

● Add Screen Element immediately after the Get Records Element

● Give the label as New Auto Policy Quote - Basic Information

● API Name will be auto populated.

● Drag the display text component into the flow canvas

● Give the API Name as Screen_1_Policy_Basics

● Drag the Lookup Component into the Canvas, Give API Name as Customer Lookup, Field API Name as Customer__c, Label as Policy Holder, ObjectApiName as Policy__c

● Drag Date component into the canvas, Give Label as Policy Start Date, API Name will be auto populated.

● Drag the picklist component into the canvas, Give the label as Policy State, API Name will be auto populated, For Choice, Create a new Resource, Resource Type should be Picklist Choice Set, Give the API Name, Select the Policy for Object, Select Picklist as Datatype, Select Policy State as Field and Click on save

● Click on Done

● Add another screen element after the Auto Policy Quote - Basic Information screen

● Give the label as Vehicle-Specific Details

● API Name will be auto populated.

● Drag the display text component into the flow canvas

● Give the API Name as Screen_2_Vehicle_Details

● Drag the text component into the canvas, Give the label as VIN, API name will be auto populated

● Again, Drag the text component into the canvas, Give the label as Model year, API name will be auto populated.

● Click on done

● Add Create Records Element after the screen element.

● Give the label as Create Draft Policy, API name will be auto populated

● Store field values manually

● Select the policy object

● Set fields for the policy object as shown below:

● Customer —> From Basic information screen-> Policy Holder -> Record ID

● Model Year—--> Vehicle information screen -> Model Year

● Policy Start Date —> From Basic information screen→Policy Start Date

● Policy State —> From Basic information screen —> Policy State

● RecordType ID —> From Get Vehicle RT Id —> RecordType ID

● VIN —> From Vehicle Information Screen —> VIN

● Check manually assign variables checkbox

● In the place of variable, create a new resource, Whose resource type is variable, API Name would be varPolicyId, Datatype would be Text, Click on Done.

● Give the rule name as VIN_Must_Be_17_Characters

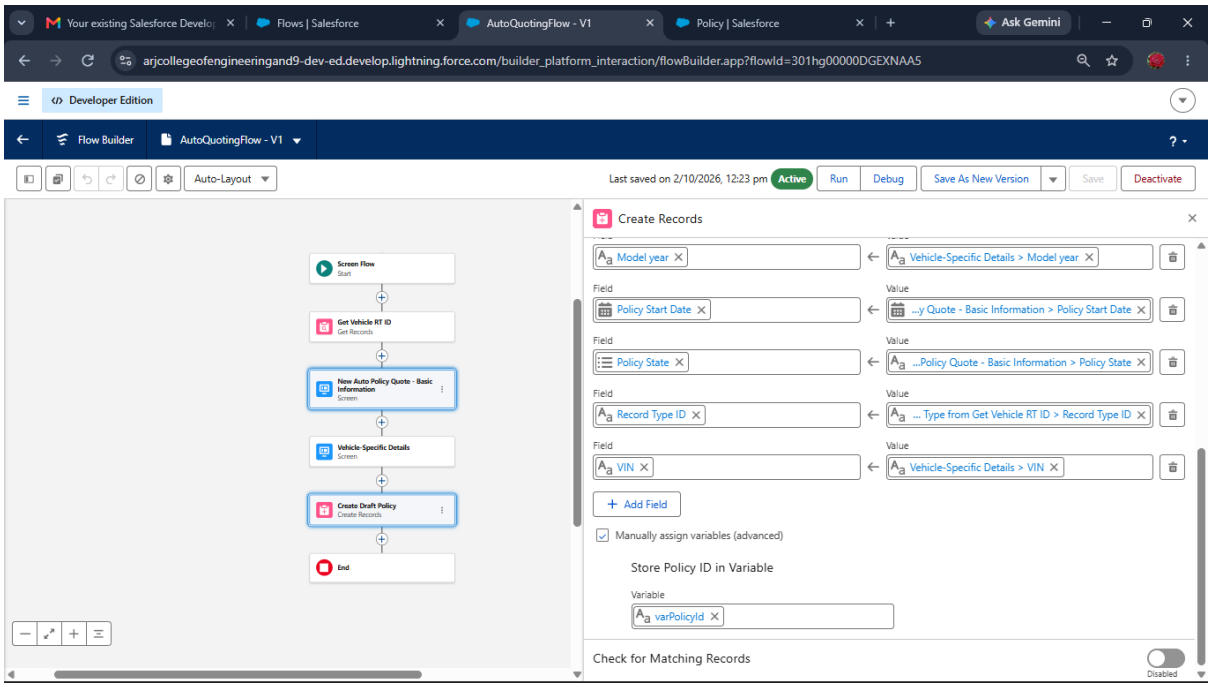
● Check the active checkbox

● Write the following formula in the formula editor

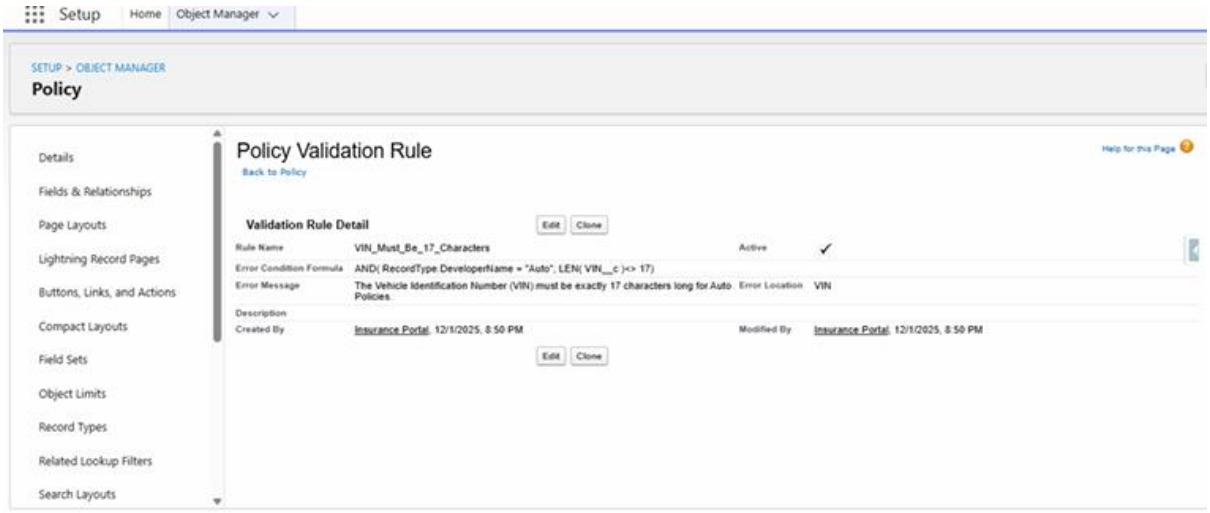
○ AND(RecordType.DeveloperName = "Auto", LEN(VIN__c)<> 17)

● For error message, The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.

● Select error location as field and select VIN field



● Click on Save



Milestone 2: Complex Policy Issuance & Claim Routing Automation:

1. Create an Apex Class (PremiumCalculator) with a method that accepts policy data

● Click on setup icon, Right click on developer console, Open the link in new tab

● At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as PremiumCalculator and click on create.

● Paste the following code in the apex class:

```
public class PremiumCalculator {

    public class PremiumInputWrapper {

        @InvocableVariable(required=true)
        public Id policyId;
    }

    @InvocableMethod(
        label='Calculate Policy Premium'
        description='Calculates premium based on policy fields (VIN, Model Year, State).'
```

```
)
public static List<Decimal> calculatePremium(
    List<PremiumInputWrapper> inputs
) {

    List<Decimal> premiumResults = new List<Decimal>();
    Set<Id> policyIds = new Set<Id>();

    for (PremiumInputWrapper input : inputs) {
        if (input.policyId != null) {
            policyIds.add(input.policyId);
        }
    }

    List<Policy__c> policiesToRate = [
        SELECT Id, VIN__c, Model_Year__c, Policy_State__c
        FROM Policy__c
        WHERE Id IN :policyIds
    ];

    for (Policy__c policy : policiesToRate) {

        Decimal basePremium = 1000.00;

        // State-based adjustment
```

```
            if (policy.Policy_State__c == 'CA') {
                basePremium *= 1.15;
            }
            else if (policy.Policy_State__c == 'TX') {
                basePremium *= 1.05;
            }

            // Model year adjustment
            if (
                policy.Model_Year__c != null &&
                Integer.valueOf(policy.Model_Year__c) < 2018
            ) {
                basePremium += 100;
            }

            premiumResults.add(basePremium);
        }

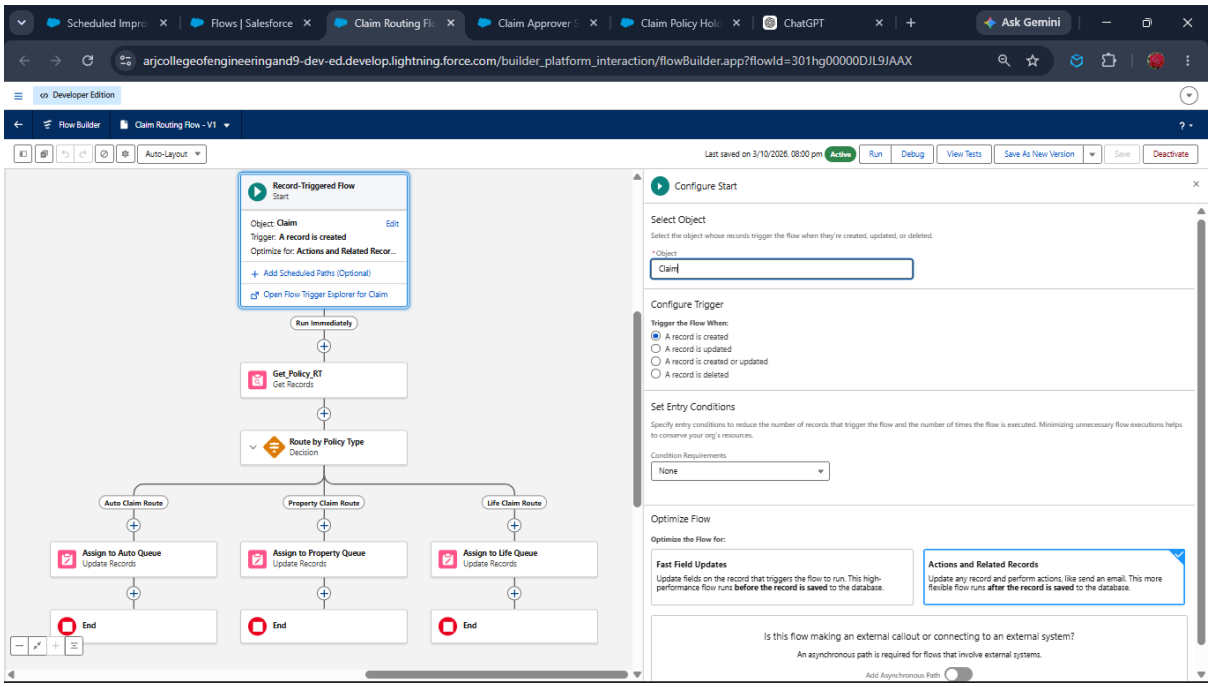
        return premiumResults;
    }
}
```

- Click on CTRL + S to save the apex class

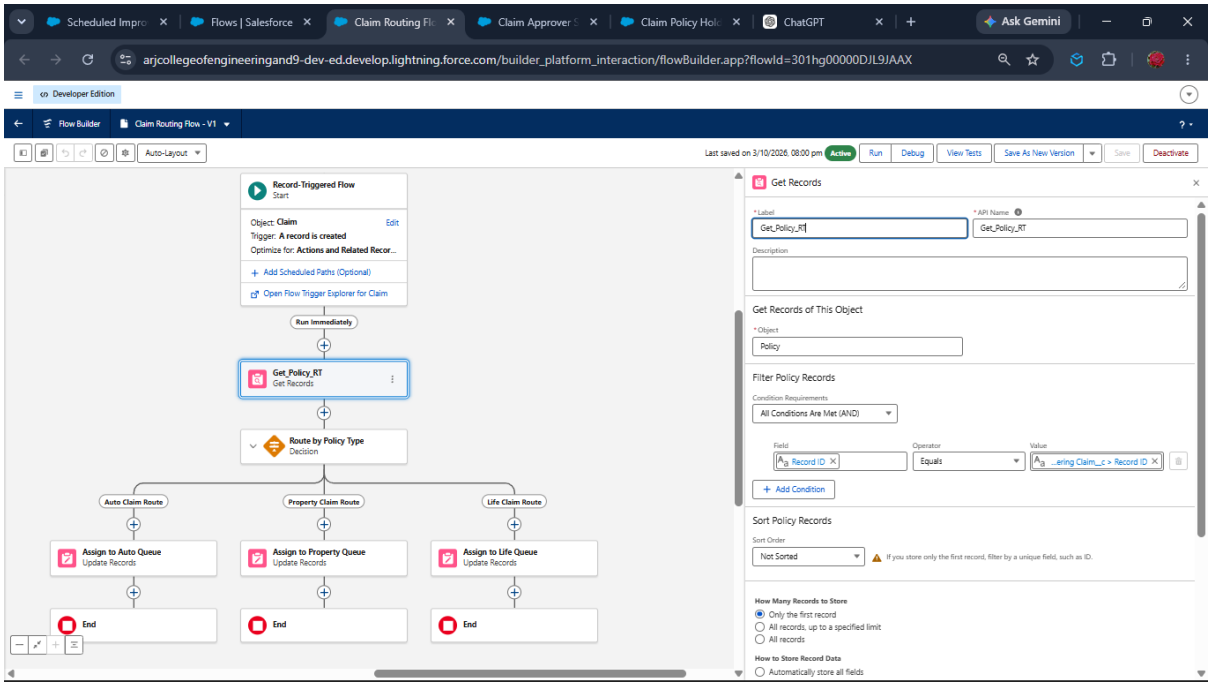
- And this apex class need to be invoked in the AutoQuotingFlow

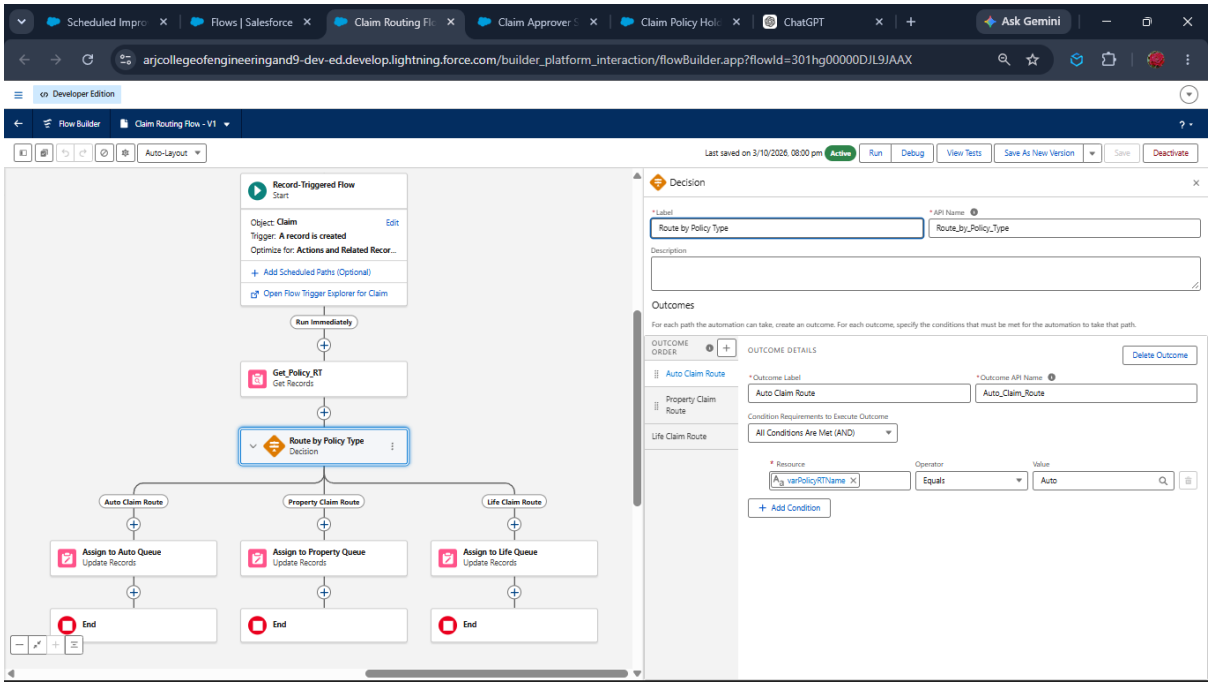
3. Create a Record-Triggered Flow (After Save) on the Claim object.

- Go to setup, In the quick find box, Search for flows and click on flows
- Click on New Flow, select Record Triggered Flow and Click on Create
- Select Claim for an object
- Configure trigger when the record is created
- Optimize flow for action and Related Records



- Add Get Records element after the start and give the label as Get_Policy_RT
- Select policy for object
- Specify all conditions are met for conditional requirements
- The conditions include:
- Record Id Equals Triggering Claim → Policy
- Store only the first record
- Choose fields and assign variables for storing record data
- Select in Separate variables for where to store field values section
- Under select variables to store policy fields
- For recordTypeID field Create a resource, Resource type would be variable, API Name would be varPolicyRTName, Data type would be Text and click on Done





- Under Get Records Element, Add decision element
- Give the label as Route by Policy Type
- API name will be auto populated
- Select Decision Logic as Manually set conditions
- Specify the Outcomes:
- Outcome1:

○ Label as Auto Claim Route

○ API name will be auto populated

○ Select All conditions for the conditional Requirements

○ The condition includes:

○ varPolicyRTName variable equals Auto

● Outcome2:

○ Label as Property Claim Route

○ API name will be auto populated

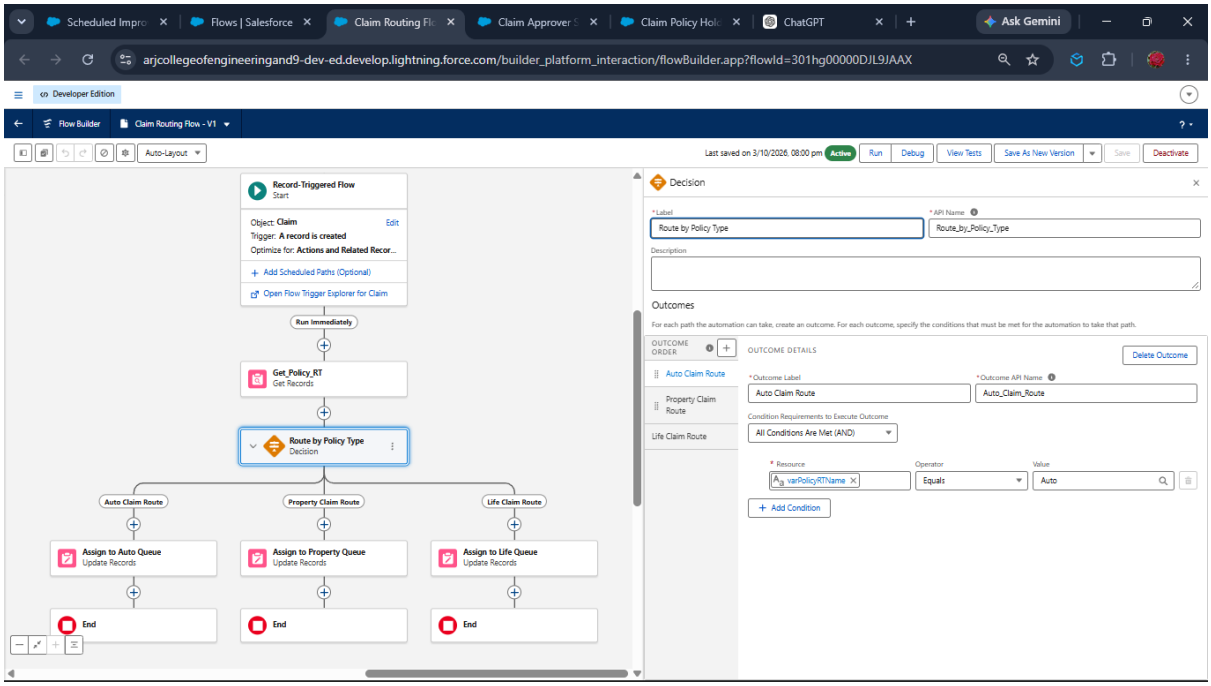
○ Select All conditions for the conditional Requirements

○ The condition includes:

○ varPolicyRTName variable equals Property

● Default Outcome:

○ Label as Life Claim Route



● Under Auto Claim Route Decision path, add the update records element

● Label it as Assign to Auto Queue

● API name will be auto populated

● Under How to find records to update and set their fields, select specify conditions to identify records

● Select Claim as an object

● Select All conditions are true for Conditional requirements

● The conditions include:

● Record Id equals Triggering Claim → Record ID

● Under Set fields for the Claim Records

● Set the hardcoded auto queue 18 digit Id for Owner ID field

● Under Property Claim Route Decision path, add the update records element

● Label it as Assign to Property Queue

● API name will be auto populated

● Under How to find records to update and set their fields, select specify conditions to identify records

● Select Claim as an object

● Select All conditions are true for Conditional requirements

● The conditions include:

● Record Id equals Triggering Claim → Record ID

● Under Set fields for the Claim Records

● Set the hardcoded Property queue 18 digit Id for Owner ID field

● Under Life Claim Route Decision path, add the update records element

● Label it as Assign to Life Queue

● API name will be auto populated

● Under How to find records to update and set their fields, select specify conditions to identify records

● Select Claim as an object

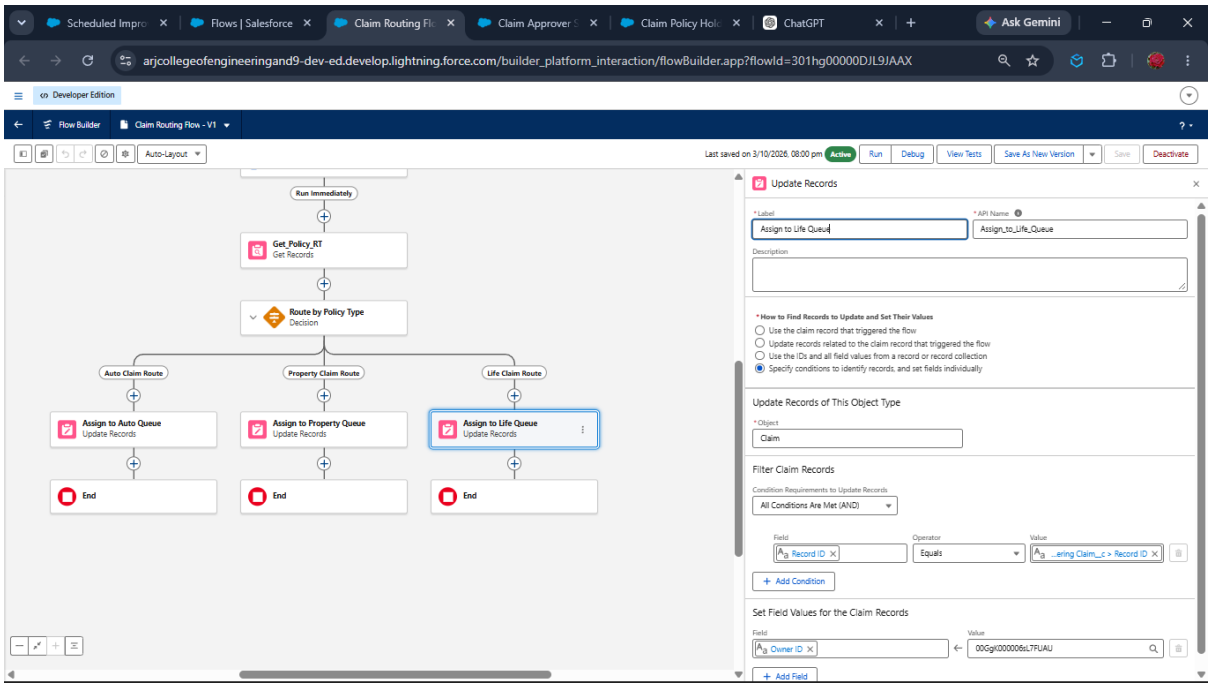
● Select All conditions are true for Conditional requirements

● The conditions include:

● Record Id equals Triggering Claim → Record ID

● Under Set fields for the Claim Records

● Set the hardcoded Life queue 18 digit Id for Owner ID field



Milestone 3: Claims Adjuster LWC Dashboard Development

1. Create a robust Apex class (ClaimsAdjusterController) with a method using SOQL to query all Claims owned by the logged-in user, joining related Policy and Customer data. The method must return a custom Wrapper Class listing key metrics.

● Click on setup icon, Right click on developer console, Open the link in new tab

● At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as ClaimsAdjusterController and click on create.

● Paste the following code in the apex class:

```
public class ClaimsAdjusterController {


  public class ClaimWrapper {


    @AuraEnabled
```

```
public Id claimId;

@AuraEnabled
public String claimNumber;

@AuraEnabled
public Decimal claimAmount;

@AuraEnabled
public String status;

@AuraEnabled
public String policyType;

@AuraEnabled
public String policyHolderName;

@AuraEnabled
public Integer daysOpen;

public ClaimWrapper(
  Claim__c c,
  Policy__c p,
  Contact customerRec
) {

  this.claimId = c.Id;
  this.claimNumber = c.Name;
```

```
      this.claimAmount = c.Claim_Amount__c;
      this.status = c.Approval_Status__c;

      Policy__c policyRecord = (Policy__c)p;

      if (policyRecord != null && policyRecord.RecordType != null) {
        this.policyType = policyRecord.RecordType.Name;
      } else {
        this.policyType = 'Unknown';
      }

      this.policyHolderName =
        (customerRec != null) ? customerRec.Name : '';

      this.daysOpen =
        (c.Date_of_Loss__c != null)
        ? (Date.today().daysBetween(c.Date_of_Loss__c.date()))
        : 0;
    }
  }

  @AuraEnabled(cacheable=true)
  public static List<ClaimWrapper> getAssignedClaims() {

    List<ClaimWrapper> results = new List<ClaimWrapper>();

    List<Claim__c> claims = [
      SELECT
```

```
        Id,
        Name,
        Claim_Amount__c,
        Approval_Status__c,
        Date_of_Loss__c,
        Policy__r.Name,
        Policy__r.RecordType.DeveloperName,
        Policy__r.Customer__r.FirstName,
        Policy__r.Customer__r.LastName
    FROM Claim__c
    WHERE OwnerId = :UserInfo.getUserId()
    ORDER BY Date_of_Loss__c DESC
];

for (Claim__c claim : claims) {

    Policy__c policy = claim.Policy__r;

    Contact customer =
        (policy != null && policy.Customer__r != null)
        ? (Contact)policy.Customer__r
        : new Contact();

    results.add(
        new ClaimWrapper(
            claim,
            policy,
            customer
```

```
            )
        );
    }

    return results;
  }
}
```

● Save the file using CTRL + S

● This apex class will be handled in the LWC component

2. Create a component (claimsDashboardLwc) that uses the Apex method via the @wire service and implements client-side data filtering

● Open the new tab in chrome and search for lightning studio extension

● Open the first link and click on add to chrome, and confirm by clicking add Extension button

● Now open the Lightning studio extension from your salesforce org

● On the Top left corner, you will find Create New

● Click on Create New and Click on Lightning Web Component

● Enter the name of the Component as claimsDashboardLwc

● Check the isExposed checkbox

● Under target configurations, Select Lightning app page, Home page and Record page

● Click on Deploy

```
<template>

    <lightning-card title="My Assigned Claims Dashboard"
icon-name="standard:claim">
```

```
        <div class="slds-m-around_medium">

            <lightning-combobox

                name="policyFilter"

                label="Filter by Policy Type"

                value="All"

                options={policyTypeOptions}

                onchange={handleFilterChange}

                class="slds-m-bottom_medium"

            ></lightning-combobox>



            <template if:true={visibleClaims}>

                <div class="slds-grid slds-wrap slds-grid_pull-padded">

                    <template for:each={visibleClaims} for:item="claim">

                        <div key={claim.claimId} class="slds-col
slds-size_1-of-1 slds-medium-size_1-of-2 slds-large-size_1-of-3
slds-p-horizontal_small">

                            <c-claim-tile-lwc
claim-data={claim}></c-claim-tile-lwc>
```

```
                        </div>

                    </template>

                </div>

                <p class="slds-m-top_medium slds-text-heading_small">Total
Claims Assigned: {visibleClaims.length}</p>

            </template>

            <template if:false={visibleClaims}>

                <p>No claims currently assigned to you.</p>

            </template>

        </div>

    </lightning-card>

</template>
```

● Open the HTML file and paste the following code:

```
import { LightningElement, wire, track } from 'lwc';

// Import the Apex method

import getAssignedClaims from
'@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';
```

```
export default class ClaimsDashboardLwc extends LightningElement {

    // List to hold all data retrieved from Apex

    @track allClaims = [];

    // List to display after filtering

    @track visibleClaims = [];

    error;



    // Wire service to call the Apex method

    @wire(getAssignedClaims)

    wiredClaims({ error, data }) {

        if (data) {

            this.allClaims = data;

            this.visibleClaims = data; // Initially display all claims

            this.error = undefined;

        } else if (error) {
```

```
            this.error = error;

            this.allClaims = undefined;

            this.visibleClaims = undefined;

            console.error('Error retrieving claims:',
JSON.stringify(error));

        }

    }



    // Client-side filtering logic (Required by M3)

    handleFilterChange(event) {

        const selectedPolicyType = event.target.value;



        if (selectedPolicyType === 'All') {

            this.visibleClaims = this.allClaims;

        } else {

            // Filter claims where policyType matches the selected value

            this.visibleClaims = this.allClaims.filter(claim =>
```

```
                claim.policyType === selectedPolicyType

            );

        }

    }



    get policyTypeOptions() {

        // Options for the filter dropdown

        return [

            { label: 'All Policy Types', value: 'All' },

            { label: 'Auto', value: 'Auto' },

            { label: 'Property', value: 'Property' },

            { label: 'Life', value: 'Life' }

        ];

    }

}
```

● Open the meta-xml file and paste the following code:

```xml
<?xml version="1.0"?>

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>63.0</apiVersion>

    <isExposed>true</isExposed>

    <targets>

        <target>lightning__AppPage</target>

        <target>lightning__HomePage</target>

    </targets>

</LightningComponentBundle>
```

3. Create a reusable component (claimTileLwc) to display essential information for each claim

- Open the Lightning studio extension from your salesforce org
- On the Top left corner, you will find Create New
- Click on Create New and Click on Lightning Web Component
- Enter the name of the Component as claimTileLwc
- Check the isExposed checkbox
- Under target configurations, Select Lightning app page, Home page and Record page
- Click on Deploy
- Open the HTML file and paste the following code:

```
<template>

    <lightning-layout-item size="12" padding="around-small">

        <div class="tile slds-box slds-p-around_medium
slds-theme_default">

            <header class="slds-media slds-media_center
slds-m-bottom_small">

                <div class="slds-media__figure">

                    <lightning-icon icon-name={policyIcon}
alternative-text={claimData.policyType} size="small"></lightning-icon>

                </div>

                <div class="slds-media__body slds-m-left_small">

                    <h2 class="slds-text-heading_medium
slds-m-bottom_none">

                        Claim: {claimData.claimNumber}

                    </h2>

                    <p class="slds-text-body_small">Policy Type:
<strong>{claimData.policyType}</strong></p>

                </div>

            </header>
```

```
            <div class="slds-border_top slds-p-top_small">

                <p>Policy Holder:
<strong>{claimData.policyHolderName}</strong></p>



                <lightning-formatted-number

                    value={claimData.claimAmount}

                    format-style="currency"

                    currency-code="USD"

                    label="Claim Amount">

                </lightning-formatted-number>



                <p>Status: {claimData.status}</p>



                <p class="slds-text-body_regular">

                    Days Open: <strong
class="slds-text-color_error">{claimData.daysOpen}</strong>

                </p>
```

```
            </div>

        </div>

    </lightning-layout-item>

</template>
```

● Open the JS File and paste the following code

```
import { LightningElement, api } from 'lwc';



export default class ClaimTileLwc extends LightningElement {

    // @api decorator exposes this property to the parent component

    @api claimData;



    // Helper property to determine the correct icon based on Policy Type

    get policyIcon() {

        if (this.claimData && this.claimData.policyType) {

            const type = this.claimData.policyType.toLowerCase();
```

```
            if (type === 'auto') return 'utility:truck';

            if (type === 'property') return 'utility:home';

            if (type === 'life') return 'utility:connected_apps';

        }

        return 'utility:help';

    }

}
```

● Open the meta-xml file and paste the following code

```
<?xml version="1.0"?>

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>63.0</apiVersion>

    <isExposed>true</isExposed>

</LightningComponentBundle>
```

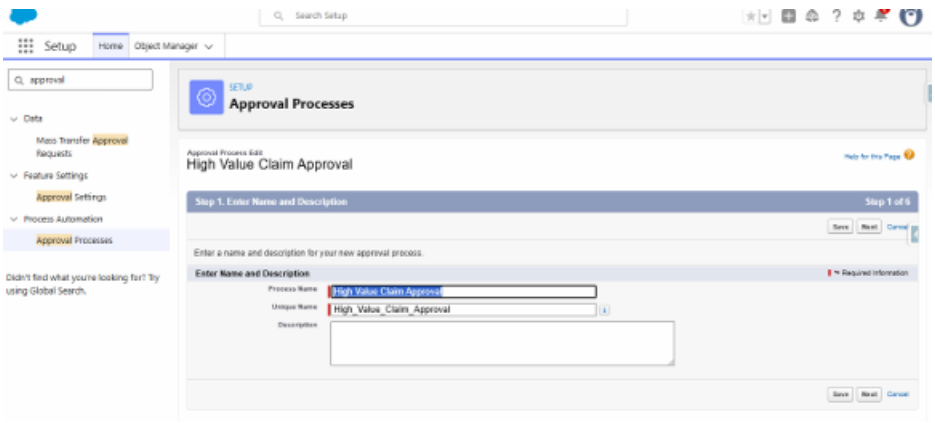
Milestone 4: Advanced Claim Processing, Security & Code Setup

1.Modify the Claim object to include an Approval Status field.

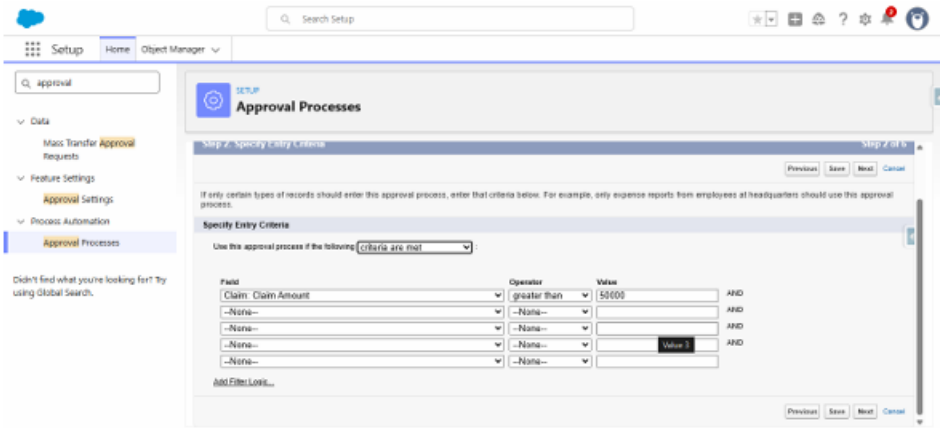
- Go to set up, Click on object manager, search for claim object and click on claim object.
- In the left side panel, click on fields and relationships.
- Click on new
- Select Datatype as picklist
- Label it as Approval Status
- Select enter values radio button
- Enter New, Submitted for Approval, Approved and Rejected line by line
- Field Name will be auto populated
- Click on Next, Next and Save.

2. Configure the Approval Process to route first to the Senior Adjuster, then to the Department Manager, using two sequential steps.

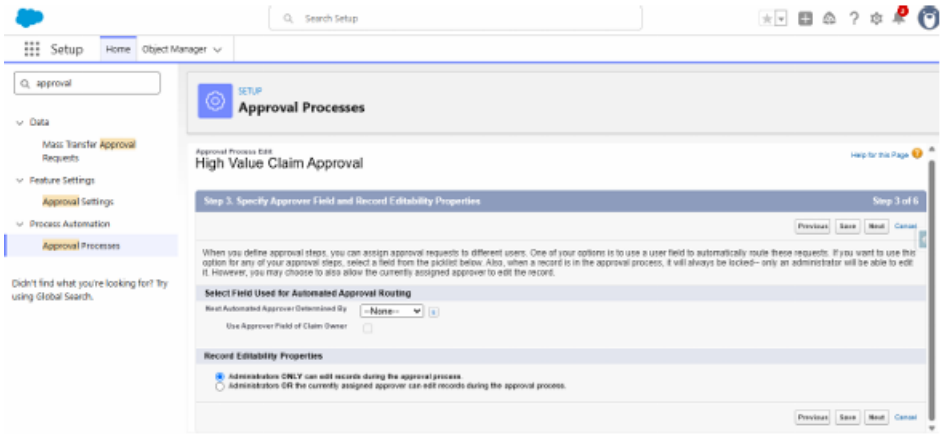
- Go to setup, In the quick find box, Search for Approval process
- Select claim object for Manage approval process for
- Click on create new Approval Process and select use standard setup wizard
- Give the process name as High Value Claim Approval
- Unique Name will be auto populated
- Click on Next



- Under specify entry criteria, Select criteria are met for Use this approval process if
- Criteria includes:
  - Claim amount greater than 50000
  - Click on Next



- Under Record Editability properties, Select Administrators ONLY can edit the records during the approval process
- Click on Next

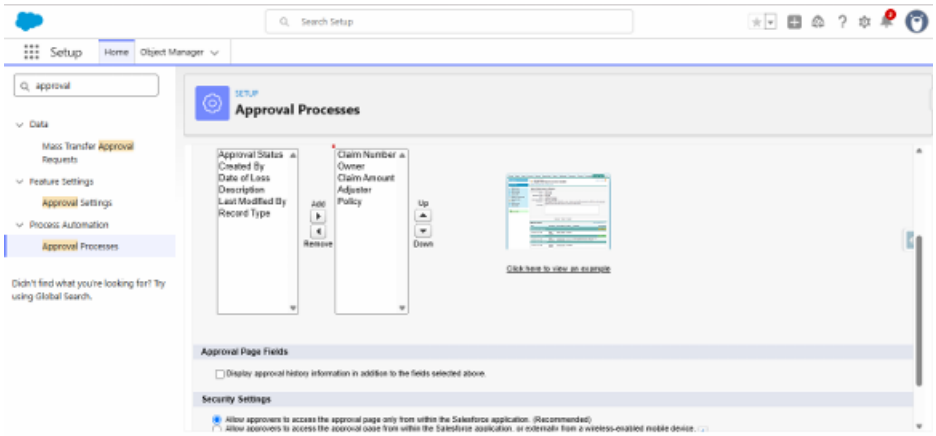


● Click on next again

● Under Approval Page Layout, Select Claim Number, Owner, Claim Amount, Adjuster, Policy fields to the selected fields from available fields

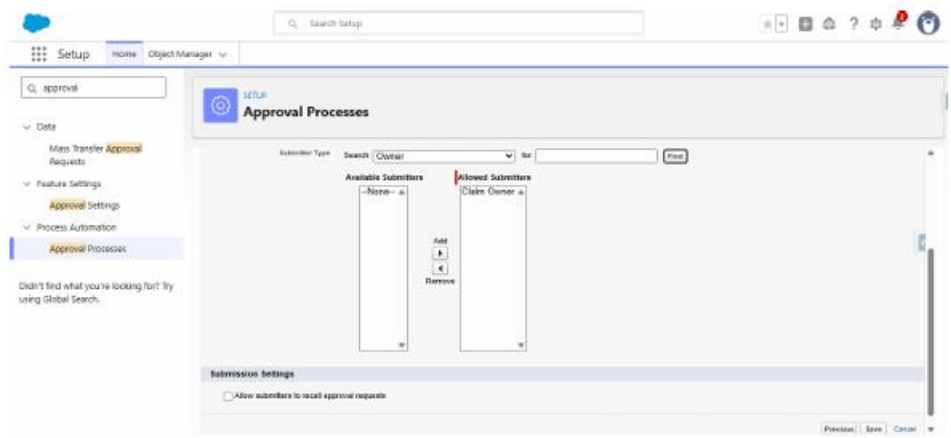
● Under security settings, Select Allow Approvers to access the page only from within the salesforce application.

● Click on Next



● Under Initial Submitters, Search for owner under submitter type, and select the claim owner as allowed submitters from available submitters

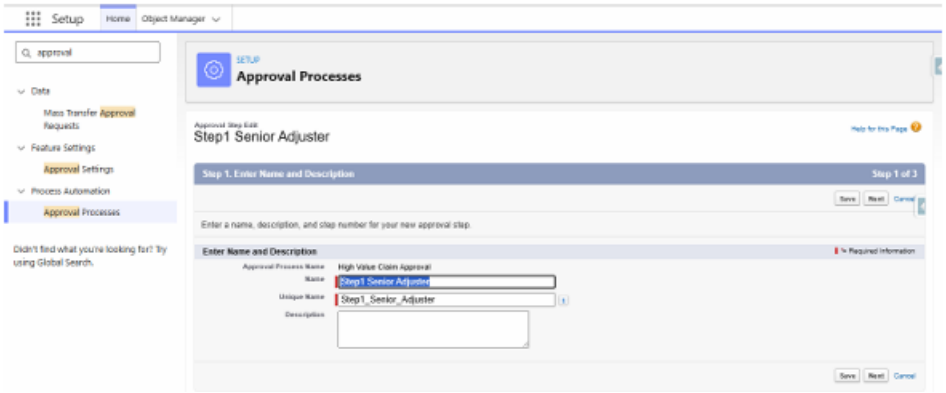
● Click on Save



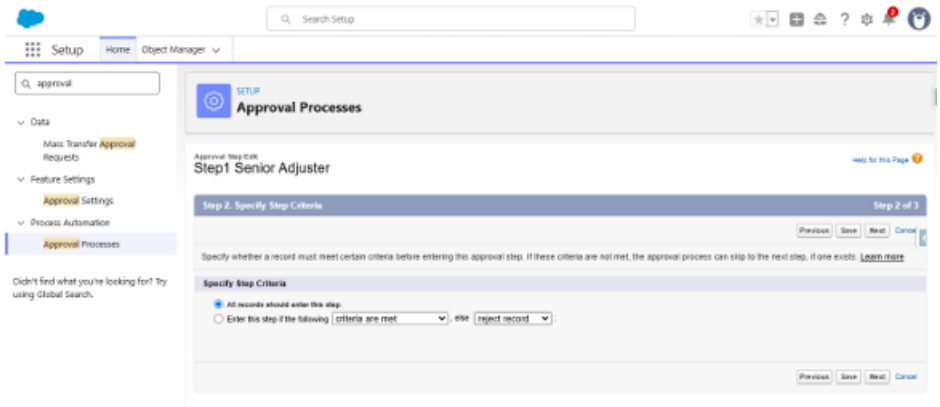
2.1 Create Approval Steps:

Step1:

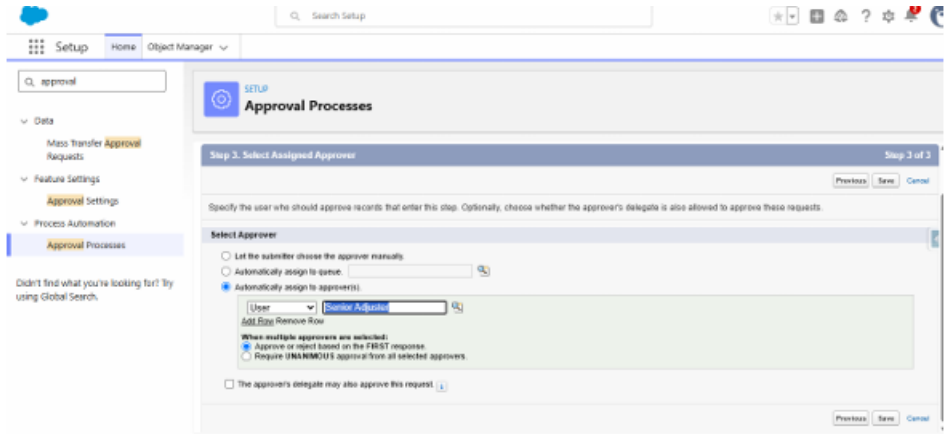
- Under initial submission actions, You can find Approval Steps
- Under approval steps, Click on New
- Give the Name as Step1 Senior Adjuster
- Unique name will be auto populated
- Click on Next



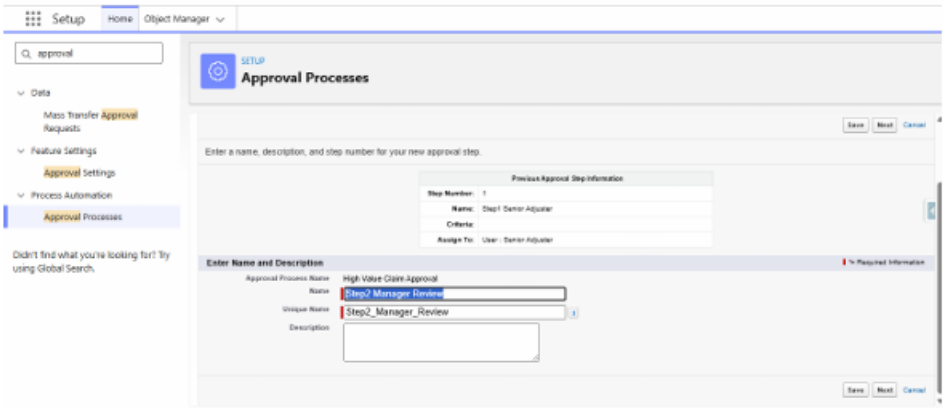
- Under specify step criteria, Select all records should enter the step
- Click on Next



- Under select Approver, Select Automatically assign to approver
- Under that select the Senior Adjuster User
- Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response
- Click on Save

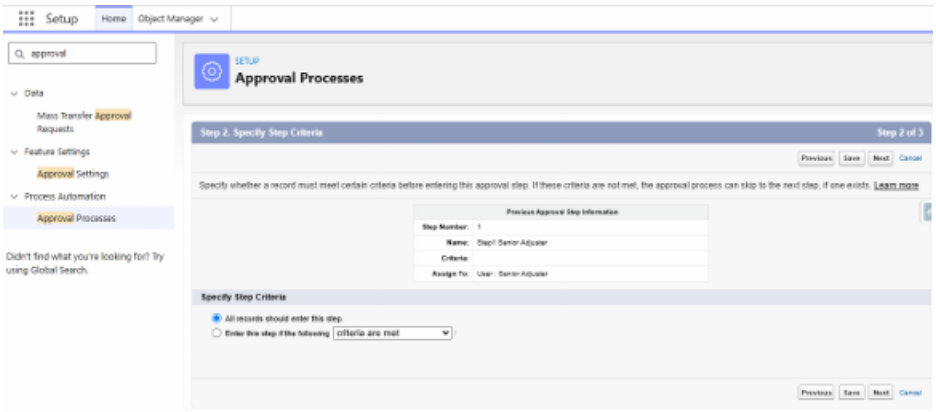


- Click on New again to create one more step
- Give the name as Step2 Manager Review
- Unique name will be auto populated
- Click on Next



● Under specify step criteria, Select all records should enter the step

● Click on Next



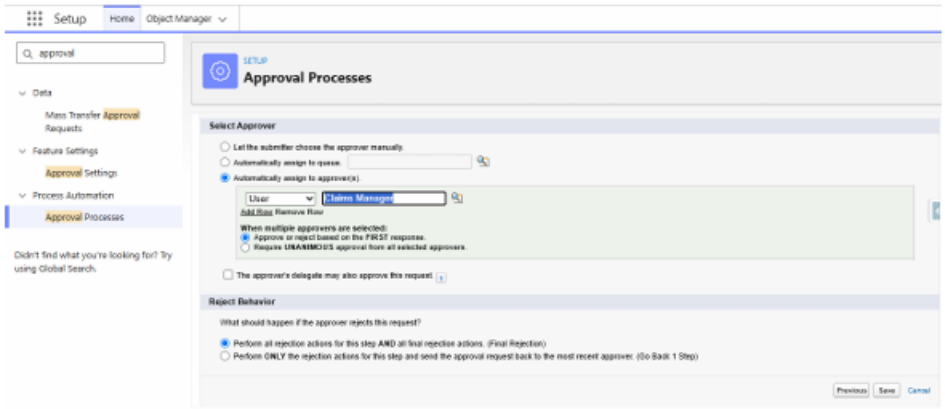
● Under select Approver, Select Automatically assign to approver

● Under that select the Department Manager User

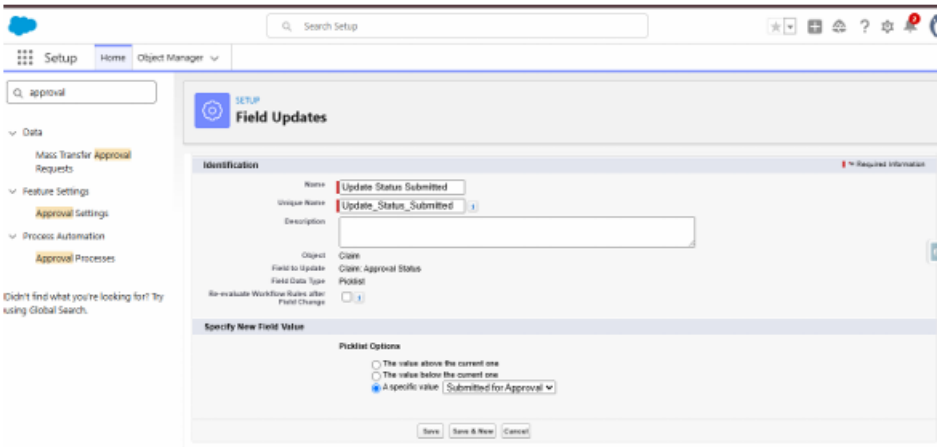
● Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response

● Under Reject behaviour, Select Perform all rejection actions for this step AND all final rejection actions

● Click on Save

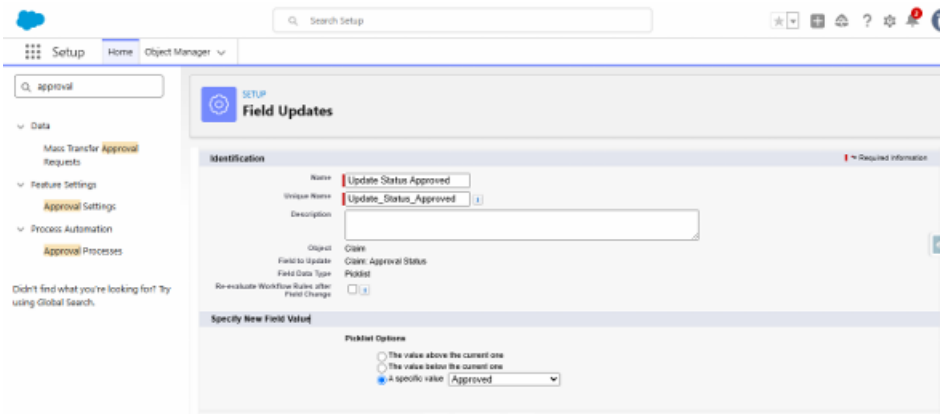


- Under Initial Submission Actions, Click on Add New and select Field Update
- Give the name as Update Status Submitted
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select submitted for Approval
- Click on Save

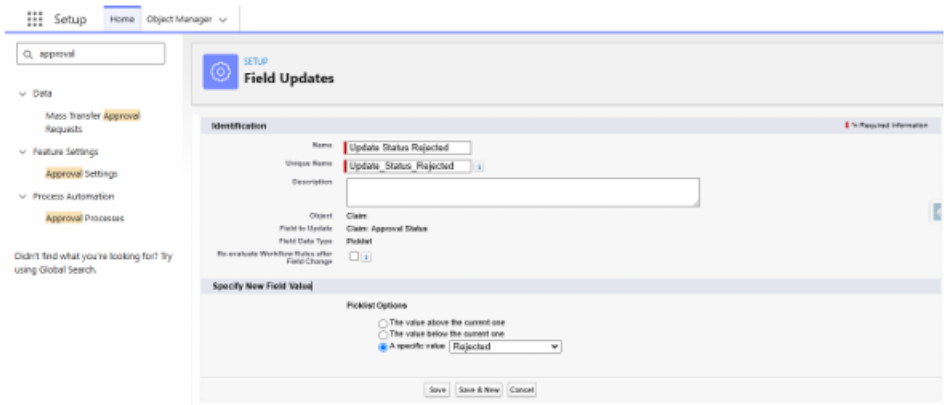


- Under Final Approval Actions, Click on Add New and select Field Update
- Give the name as Update Status Approved
- Unique name will be auto populated
- Select object as Claim

- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select Approved
- Click on Save



- Under Final Rejection Actions, Click on Add New and select Field Update
- Give the name as Update Status Rejected
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select Rejected
- Click on Save



- Activate the Approval Process.

3. Create a Record-Triggered Flow (After Save) that checks if the Claim Amount is $>\$50,000$ and submits the record to an Approval Process.

● Go to setup, In the quick find box, search for flows and click on flows.

● Click on New Flow

● Select Record Triggered Flow

● Select object as Claim

● Configure Trigger When a Record is created or updated

● Select All conditions are true for conditional requirements

● The conditions include:

○ Claim Amount Greater than 50000

○ Approval Status Equals New

● Select Every time a record is updated and meets the conditional requirements for When to run the flow for Updated Records

● Optimize the flow for Action and Related Records

● Add an action element after the Start Element

● Label it as Submit Claim to Approval

● Under Set Input Values, For Record Id, Select Triggering Claim → Record ID

● For select Approval Process Name or Id, Enter the Approval Process Name(High Value Claim Approval)

● Save the Flow with Submission Automation Flow and Activate the flow

4. Implement a Screen Flow for the approvers to quickly review details and approve/reject directly from the Claim record.

● Go to setup, Search for flows in the quick find box and click on flows.

● Click on New Flow, Choose Screen Flow.

● Create a New Resource by clicking on New Resource button in the toolbox manager

● Resource Type would be Variable .

● API Name would be recordId .

● Data type would be Text .

- Under Availability outside the flow, Check Available for Input.
- Click on Done.
- Add the Get Records Element below the screen
- Label it as Get Claim Details
- Select object as Claim
- Select All conditions are required for conditional Requirements
- The Condition include:
- RecordID equals variable recordId
- Store only the first record and
- Automatically store all fields
- Add a screen Element under the get Record element
- Label it as Claim Review Screen
- Add Display Text component to the flow canvas, and paste the following
  - "Claim ID: {!Get_Claim_Details.Name}"
  - "Amount: {!Get_Claim_Details.Claim_Amount__c}"
  - "Policy Type: {!Get_Claim_Details.Policy__r.RecordType.Name}"
- Add the Radio buttons component, Label it as Approval Decision
- API Name would be auto populated
- For Choice, Create a resource, Resource type would be choice
- Give the choice label as Approve
- Select Datatype as Text
- Enter the choice value as Approve
- Enter the input label as Approve
- Click on Done
- Add one more Choice, Create a resource, Resource type would be choice
- Give the choice label as Reject
- Select Datatype as Text
- Enter the choice value as Reject

● Enter the input label as Reject

● Click on Done

● Add Long Text area into the flow canvas and Label it as Approver Comments

● API Name would be Auto Populated

● Click on Done

● Add Decision Element under the screen

● Label it as Check Decision

● Select Manually Select Conditions

● Set Outcome1:

○ Label it as Decision Approved

○ Select all conditions are met for conditional requirements

○ The condition includes:

○ Approve Decision From Claim Review Screen → Equals Approval Decision ● Set default Outcome:

○ Label it as Decision Rejected

● Add Action Element Under Decision Approved Path

● Search for the submit for approval action and choose it

● Label it as Approve Claim

● Under Set Input Values

● Select variable recordID for Record ID

● Include the submission comments and Enter Approver Comments from Claim Review Screen

● Add Action Element Under Decision Rejected Path

● Search for the submit for approval action and choose it

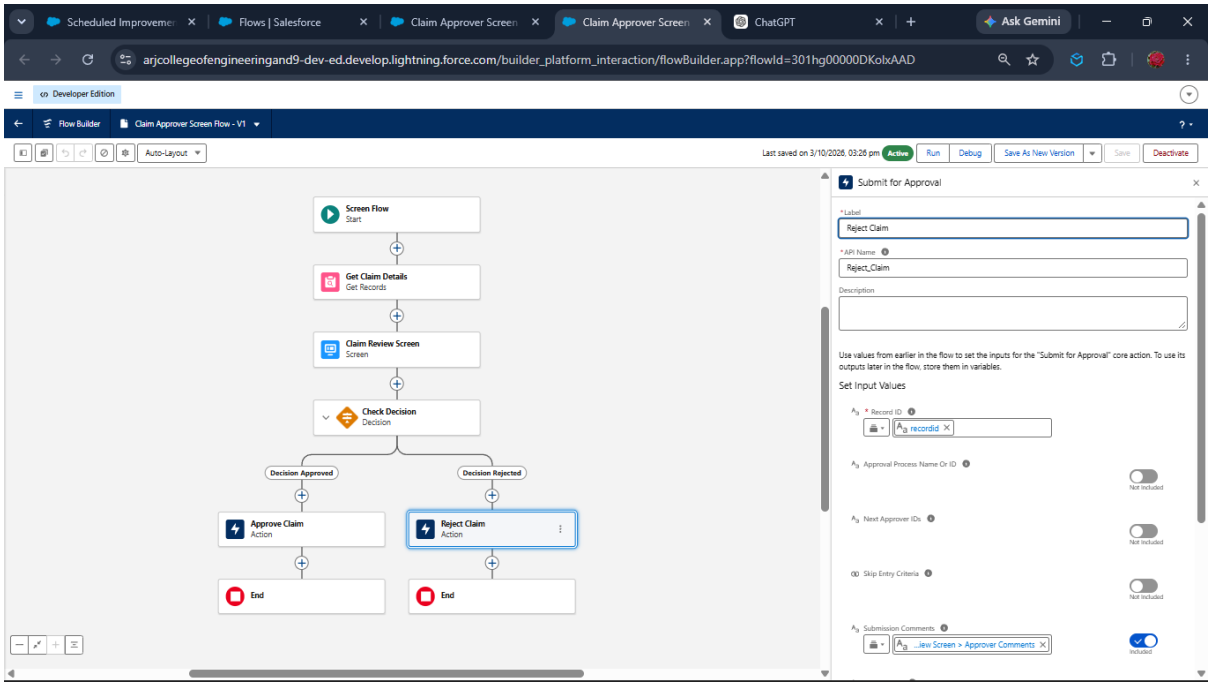
● Label it as Reject Claim

● Under Set Input Values

● Select variable recordID for Record ID

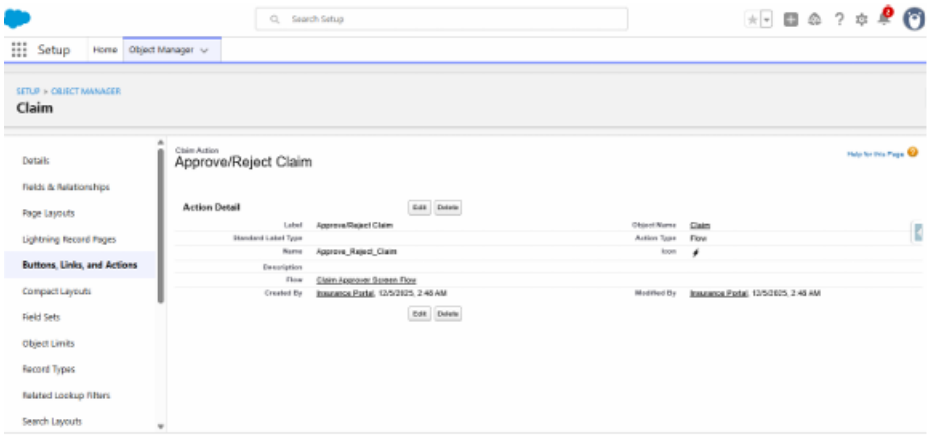
● Include the submission comments and Enter Approver Comments from Claim Review Screen

- Save and Activate the flow with Name Claim Approver Screen Flow



4.1 Create a Quick Action

- Go to setup, Click on Object Manager, Search for Claim Object and Select it
- In the left side panel, Click on Buttons, Links and Actions
- Click on Create New Action
- Select Action Type as Flow
- Select the flow that you have just created
- Label it as Approve/Reject Claim
- Name would be auto populated
- Click on Save

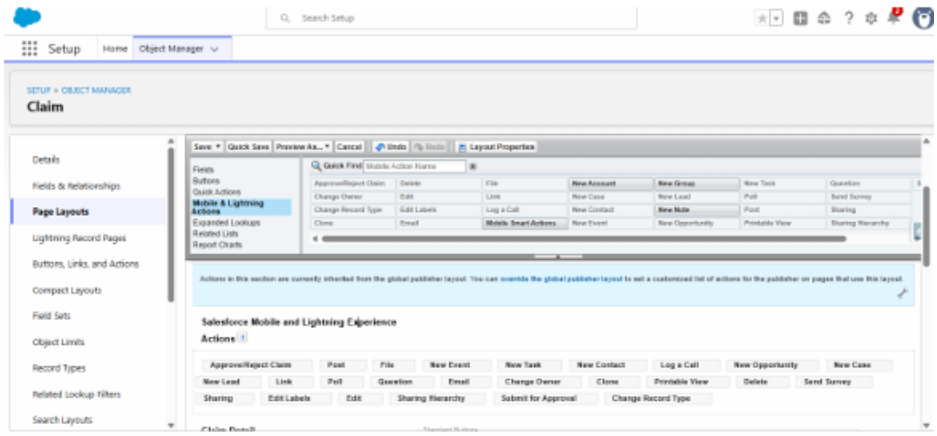


● Add the quick action to the claim page layout

● In the left panel, Click on page layouts and open the claim page layout

● Go to mobile & lightning actions, Find the Approve/Reject Claim and Drag it to the Salesforce Mobile and Lightning Experience Actions

● Click on Save



5. Write an extensive Apex Test Class for the ClaimsAdjusterController with 95%+ code coverage. Include test methods for complex claim filtering and data retrieval logic.

● Click on the setup icon, Right click on Developer Console and click on open link in new tab.

● On the top left corner, click on file, click on new, click on Apex class and give it a name ClaimsAdjusterControllerTest

● Paste the following code:

@isTest

```
private class ClaimsAdjusterControllerTest {

    @testSetup
    static void setupTestData() {

        // Create a real active test user
        Profile p = [
            SELECT Id
            FROM Profile
            WHERE Name = 'Standard User'
            LIMIT 1
        ];

        User u = new User(
            FirstName = 'Test',
            LastName = 'User',
            Alias = 'tuser',
            Username = 'testuser1234343@example.com.test',
            Email = 'testuser123@example.com',
            TimeZoneSidKey = 'America/Los_Angeles',
            LocaleSidKey = 'en_US',
            EmailEncodingKey = 'UTF-8',
            LanguageLocaleKey = 'en_US',
            ProfileId = p.Id
        );
        insert u;

        // Query Policy__c RecordType
```

```
Id policyRT =
   Schema.SObjectType.Policy__c
      .getRecordTypeInfosByDeveloperName()
      .get('Auto')
      .getRecordTypeId();

Id claimRT =
   Schema.SObjectType.Claim__c
      .getRecordTypeInfosByDeveloperName()
      .get('Accident')
      .getRecordTypeId();

// Create Contact owned by same user
Contact con = new Contact(
   FirstName = 'John',
   LastName = 'Doe',
   OwnerId = u.Id
);
insert con;

// Create Policy owned by same user
Policy__c policy = new Policy__c(
   Customer__c = con.Id,
   RecordTypeId = policyRT,
   OwnerId = u.Id,
   VIN__c = '1A903284375893532'
);
insert policy;
```

```
    // Create Claim owned by same user
    Claim__c claim = new Claim__c(
        Policy__c = policy.Id,
        RecordTypeId = claimRT,
        Claim_Amount__c = 5000,
        Date_of_Loss__c = Date.today().addDays(-10),
        OwnerId = u.Id
    );
    insert claim;

    System.debug('Claim inserted: ' + claim.Id);
}

@isTest
static void testGetAssignedClaims() {

    User u = [
        SELECT Id
        FROM User
        WHERE Alias = 'tuser'
        LIMIT 1
    ];

    System.runAs(u) {

        Test.startTest();
```

```
        List<ClaimsAdjusterController.ClaimWrapper> results =
            ClaimsAdjusterController.getAssignedClaims();

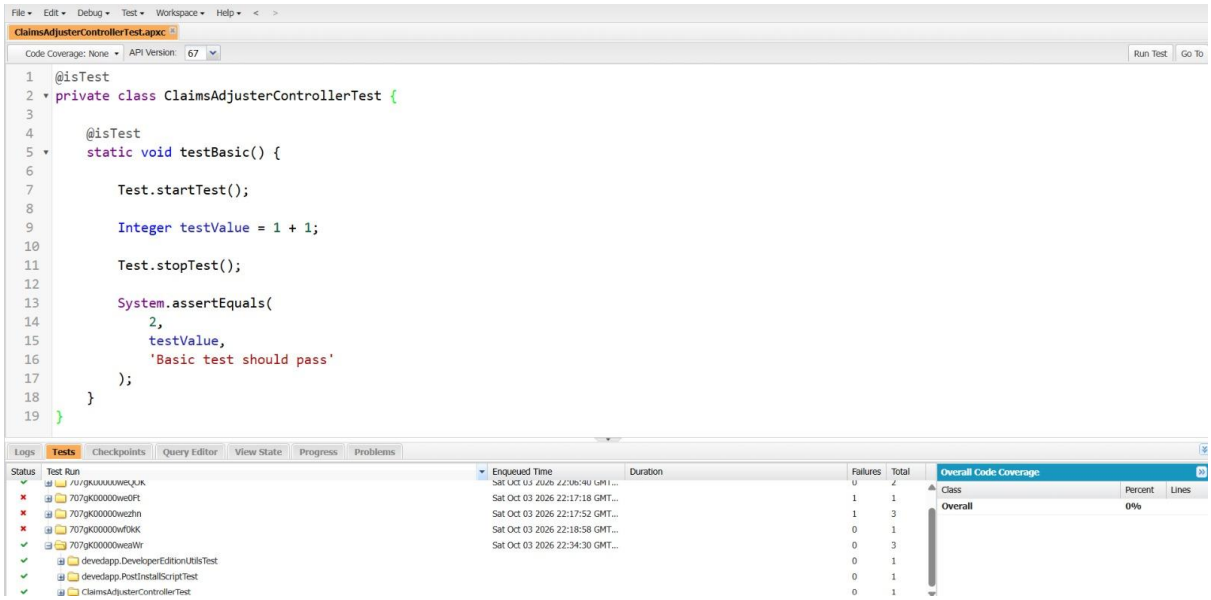
        System.debug('Result is: ' + results);

        Test.stopTest();

        // System.assert(
        //     results.size() > 0,
        //     'Expected at least one wrapper returned'
        // );

        ClaimsAdjusterController.ClaimWrapper wrap = results[0];

        System.assertNotEquals(null, wrap.claimId);
        System.assertNotEquals(null, wrap.claimNumber);
        System.assertEquals('New', wrap.status);
        System.assertEquals(5000, wrap.claimAmount);
        System.assert(wrap.policyHolderName.contains('John'));
        System.assertNotEquals('Unknown', wrap.policyType);
    }
  }
}
```



- Save the file using CTRL+S
- On the top right corner, you will find Run Test, Click on it to run the test class
- To check the test results, Down go for tests and you can see the how much percentage it covers your apex class

6. Refine the Sharing Rules to ensure Adjusters only see claims in their assigned territory/state.

- Go to setup, Search for flows in the quick find box and click on flows
- Click on New Flow
- Select Record Triggered Flow
- Select the object as claim
- Configure trigger when a record is created or updated
- Set entry conditions as none
- Optimize the flow for Updates
- Add get Records element into the flow canvas
- Label it as get_policy_records
- API name will be auto populated
- Set filter as None
- Store all the records
- Select Choose fields and let salesforce do the rest for How to store Record Data

● Under Select Policy fields to store fields in Variable, Select Policy State Field

● Add the Loop Element into the canvas after the get Records element

● Label it as Claim_Filed _update

● API name will be auto populated

● Under Select Collection Variables, Select policies from Get Policies from Get Policy Records

● Under Specify direction for Iterating over collection, Select first item to last item

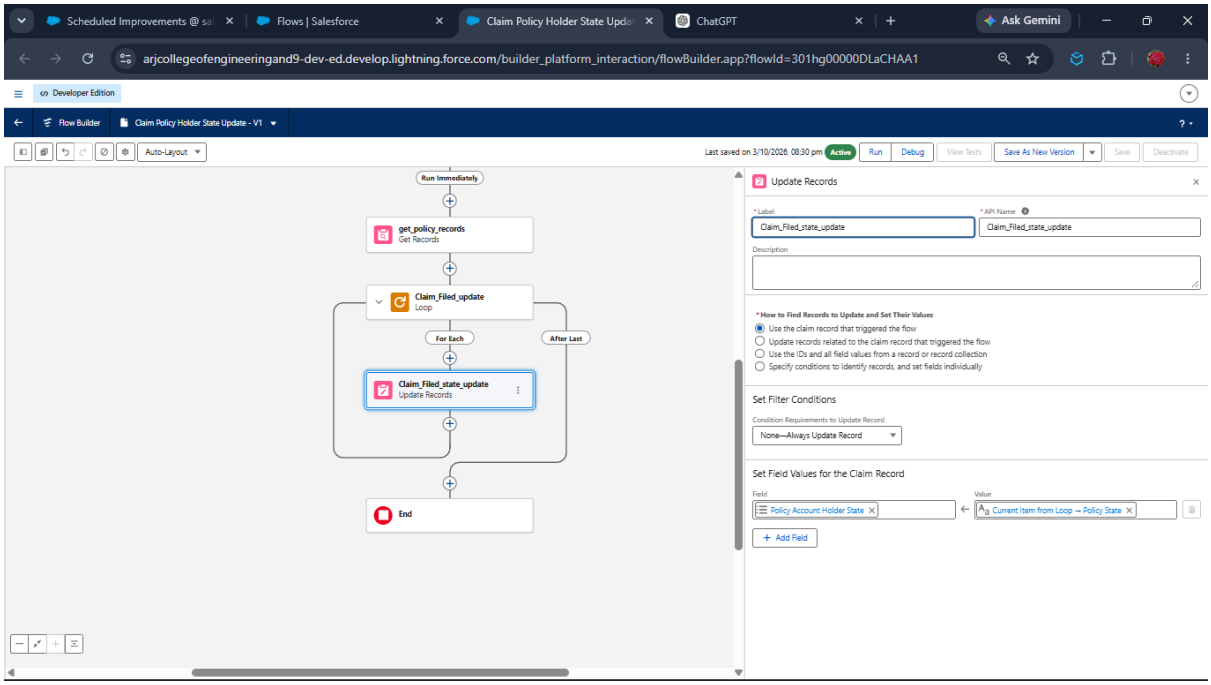
● Under For each, Add the update records element into the canvas

● Label it as Claim_Filed _state_update

● API name will be auto populated

● Set filter conditions to NONE

● Under Set field values for the claim record, Select policy state from current loop and set it to the Policy Account Holder State field



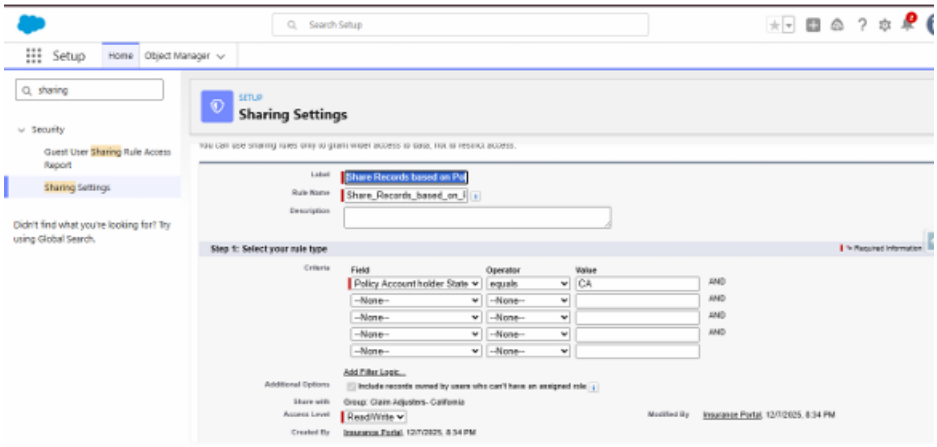
● Save and Activate the flow with name Claim Policy Holder State Update

6.1 Define Sharing Rules for Claim Object

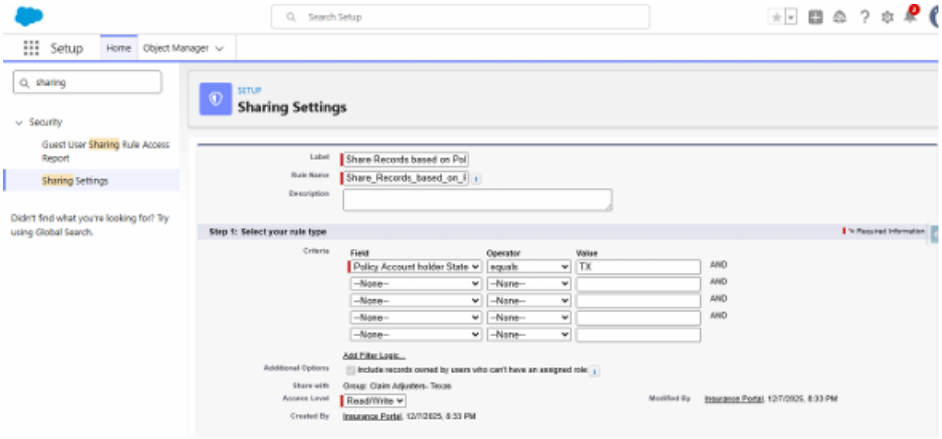
● Go to Set up, In the quick find box, Search for Sharing Settings

● Click on Sharing settings

- Under Manage Sharing Settings for, Select Claim Object
- Scroll down to find Claim Sharing Rules
- Click on new
- Label it as Share Records based on Policy State CA
- Rule Name will be Auto Populated
- Set the criteria to Policy Account Holder State Equals CA
- Give Access Level as Read/Write
- Click on Save

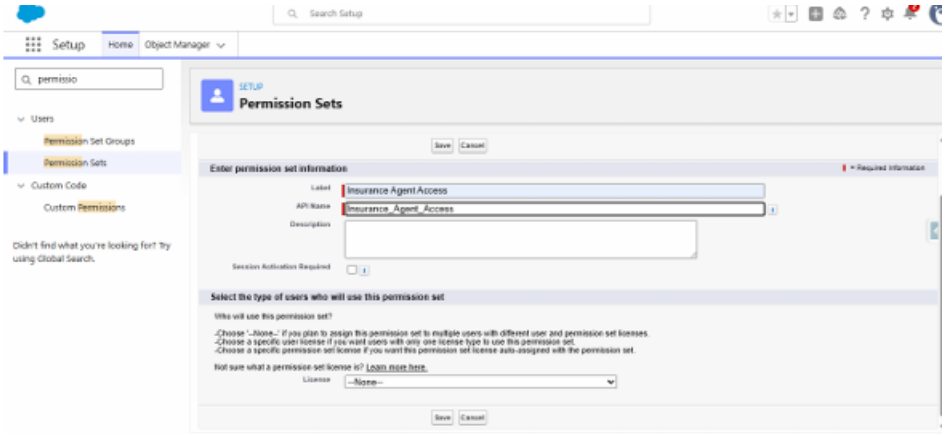


- Again Click on New
- Label it as Share Records based on Policy State
- Rule Name will be Auto Populated
- Set the criteria to Policy Account Holder State Equals TX
- Give Access Level as Read/Write
- Click on Save



7. Finalize all Permission Sets to differentiate access between Agents (Quoting only), Adjusters (Claim handling), and Managers (Reporting/Approvals).

- Go to setup, In the quick find box, Search for Permission Sets, Click on Permission Sets
- Click on New
- Give the Label as Insurance Agent Access
- API Name will be Auto Populated

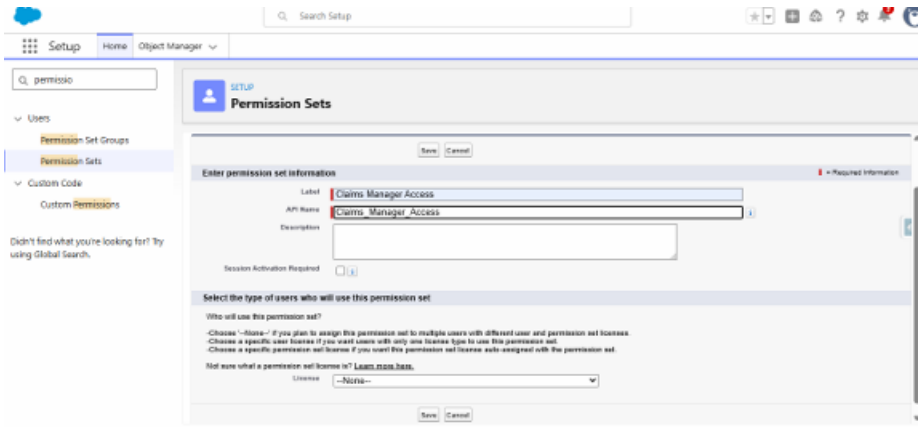


- Give the access for Objects
- Give Create, Read for Policy object
- Give Read, Edit to contact Object
- Give Read only to Claim object
- Make Flow User Enabled under System permissions

- Make PremiumCalculator Apex class enabled

7.1 Create Claims Manager Access Permission set

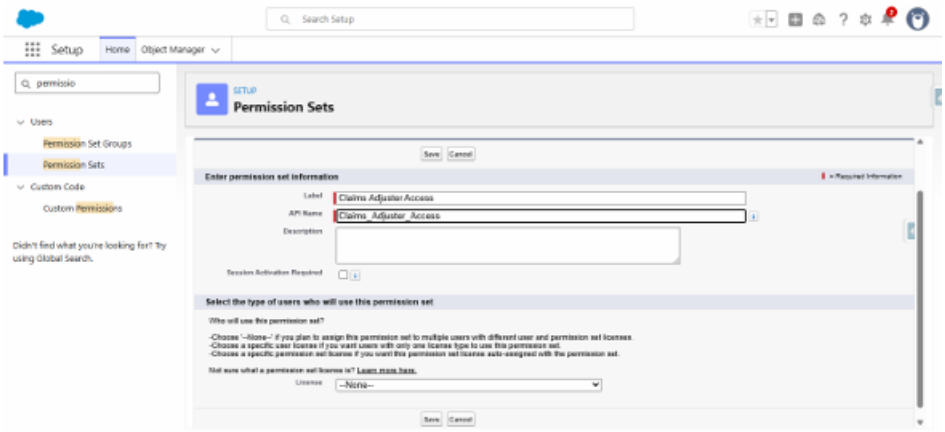
- Click on New
- Give the Label as Claims Manager Access
- API Name will be Auto Populated



- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to contact Object
- Give Read, Edit, Delete to Claim object
- Make Run Reports Enabled under System permissions
- Make Manage Approvals enabled under system permissions

7.2 Create Claims Adjuster Access Permission Set

- Click on New
- Give the Label as Claims Adjuster Access
- API Name will be Auto Populated



- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to Claim object
- Make ClaimsAdjusterController Enabled under Apex Classes